Ottawa University

Assignment 5: Advanced Monte Carlo

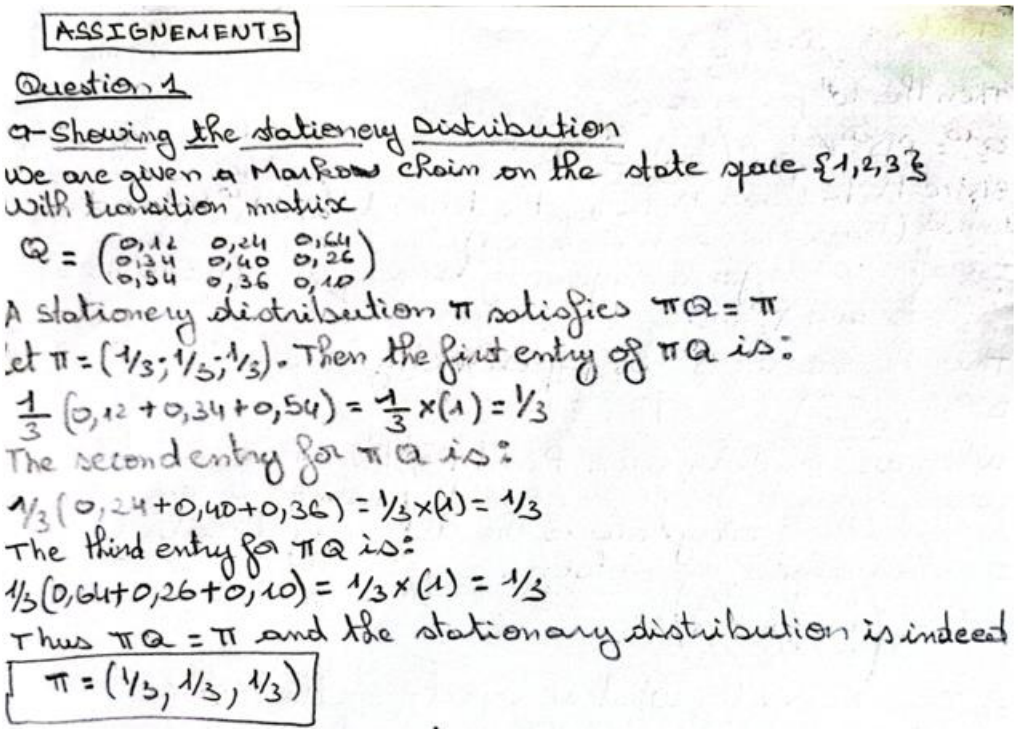
Submitted to Dr Rafal Kulik

Yann Obrou

2025-04-07

Question 1

(a) Showing the Stationary Distribution



ASSIGNEMENT 5

Question 1

a- Showing the stationary Distribution

We are given a Markov chain on the state space $\{1,2,3\}$ with transition matrix

$$Q = \begin{pmatrix} 0,12 & 0,24 & 0,64 \\ 0,34 & 0,40 & 0,26 \\ 0,54 & 0,36 & 0,10 \end{pmatrix}$$

A stationary distribution π satisfies $\pi Q = \pi$

Let $\pi = (1/3; 1/3; 1/3)$. Then the first entry of πQ is:

$\frac{1}{3}(0,12 + 0,34 + 0,54) = \frac{1}{3} \times (1) = 1/3$

The second entry for πQ is:

$1/3(0,24 + 0,40 + 0,36) = 1/3 \times (1) = 1/3$

The third entry for πQ is:

$1/3(0,64 + 0,26 + 0,10) = 1/3 \times (1) = 1/3$

Thus $\pi Q = \pi$ and the stationary distribution is indeed

$\pi = (1/3, 1/3, 1/3)$

(b) Calculating the 10th Power of Q in R

The theory tells us that for an irreducible, aperiodic Markov chain the powers Q^n converge to a matrix whose rows are all equal to the stationary distribution. Here, since the stationary distribution is $(1/3,1/3,1/3)$, we expect that Q^{10} will have every row very close to $(1/3,1/3,1/3)$.

```r
# Define the transition matrix Q
Q <- matrix(c(0.12, 0.24, 0.64,
              0.34, 0.40, 0.26,
              0.54, 0.36, 0.10), nrow = 3, byrow = TRUE)

# Function to compute the power of a matrix
matrix_power <- function(matrix, power) {
  result <- diag(nrow(matrix))  # start with the identity matrix
  for(i in 1:power) {
    result <- result %*% matrix  # multiply by Q at each iteration
  }
  return(result)
}

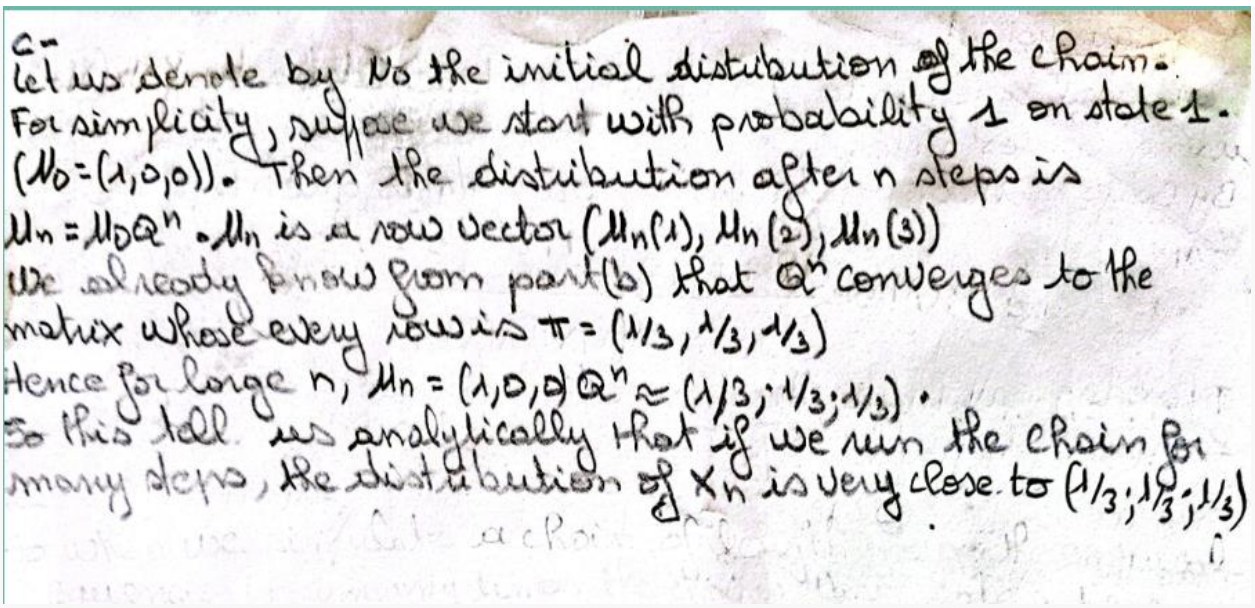
# Calculate Q^10
Q10 <- matrix_power(Q, 10)
print(Q10)
```

```
##           [,1]      [,2]      [,3]
## [1,] 0.3335765 0.3333908 0.3330327
## [2,] 0.3333013 0.3333258 0.3333729
## [3,] 0.3331222 0.3332834 0.3335944
```

b- From part a we know that $\pi = (1/3, 1/3, 1/3)$
A key property of a Markov chain with a finite state space is that 1 is an eigenvalue of Q and that the corresponding (left) eigenvector is the stationary distribution. That is, if we let π be a row vector, then $\pi Q = \pi$
Under the assumptions of irreducibility and aperiodicity (which holds for this chain), all other eingenvalues, say λ_2 and λ_3, satisfy $|\lambda_2|, |\lambda_3| < 1$
So we will write Q^{10} in terms of Eigenvalues
Using the eigendecomposition we can write: $Q = P D P^{-1}$,
where D is a diagonal matrix containing the eigenvalues

That is $D = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \lambda_2 & 0 \\ 0 & 0 & \lambda_3 \end{pmatrix}$
Then the 10th power of Q is given by
$Q^{10} = P D^{10} P^{-1} = P \begin{pmatrix} 1 & 0 & 0 \\ 0 & \lambda_2^{10} & 0 \\ 0 & 0 & \lambda_3^{10} \end{pmatrix} P^{-1}$
Since $|\lambda_2| < 1$ and $|\lambda_3| < 1$, the terms λ_2^{10} and λ_3^{10} become very small (they vanish as n increases).
In other words, for a sufficiently large power ($n = 10$ here)
$\lambda_2^{10} \approx 0$ and $\lambda_3^{10} \approx 0$
Thus the matrix D^{10} is approximately
$D^{10} \approx \begin{pmatrix} 1 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{pmatrix}$
When we multiply back P and P^{-1}, the dominant term (associated with the eigenvalue 1) produces a rank-one matrix whose every row is the stationary distribution π
In other words, we obtain.
$Q^{10} \approx \begin{pmatrix} 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \\ 1/3 & 1/3 & 1/3 \end{pmatrix}$
So This is exactly what we expect from the theory.
For an irreducible, aperiodic Markov chain, as $n \to \infty$, and even more moderately large n, such as 10 in this case), The matrix Q^n converges to a matrix with all rows equal to π

(c) Simulating a Sample of Size 100 and Comparing Proportions




```
# Function to simulate the Markov chain
simulate_markov <- function(transition_matrix, initial_state, n) {
  states <- numeric(n)
  states[1] <- initial_state
  for(i in 2:n) {
    # current_state is 1, 2, or 3; used directly for row indexing
    current_state <- states[i - 1]
    # Sample next state from the probabilities in the current state's row
    states[i] <- sample(1:3, size = 1, prob =
transition_matrix[current_state, ])
  }
  return(states)
}

# Set parameters and simulate the chain
set.seed(123)
sample_size <- 100000
chain_sample <- simulate_markov(Q, initial_state = 1, sample_size)

# Calculate proportions for each state
prop_table <- table(chain_sample) / sample_size
print(prop_table)
```

```
## chain_sample
##       1       2       3
## 0.33260 0.33548 0.33192
```

Comparison: Empirical Proportions vs. Stationary Distribution

When we simulate a chain of length $n = 100$, the **empirical frequencies** (i.e., how many times the chain visits states 1, 2, 3 divided by 100) should be *close* to $(1/3,1/3,1/3)$, but not necessarily exact—there is sampling variability. For instance, you obtained:

$$\hat{p} = (0.36,\ 0.39,\ 0.25).$$

Comparing this with $\pi = (1/3,1/3,1/3) \approx (0.33,\ 0.33,\ 0.33)$ shows a reasonable discrepancy for $n = 100$. If you increase n, the law of large numbers for Markov chains guarantees that these proportions converge to the stationary distribution in the long run.

by the Law of Large Numbers for Markov Chains:

If $\{X_t\}$ is irreducible and aperiodic with stationary distribution π, then

$$\frac{1}{n}\sum_{t=1}^{n} \mathbf{1}_{\{X_t=i\}} \xrightarrow[n\to\infty]{\text{a.s.}} \pi(i).$$

This means that the empirical proportion of time spent in state i converges almost surely to $\pi(i)$. For $n = 100$, we are simply seeing a finite-sample variation around the theoretical limit π.

Additional Observation In this simulation I increased the number of iterations from $n = 100$ to $n = 1000$, the empirical proportions for states 1, 2, and 3 converged to approximately 0.33 each. This confirms the Law of Large Numbers for Markov chains Thus, as n increases, the simulated frequencies closely approximate the stationary distribution $\pi = (1/3,1/3,1/3)$.

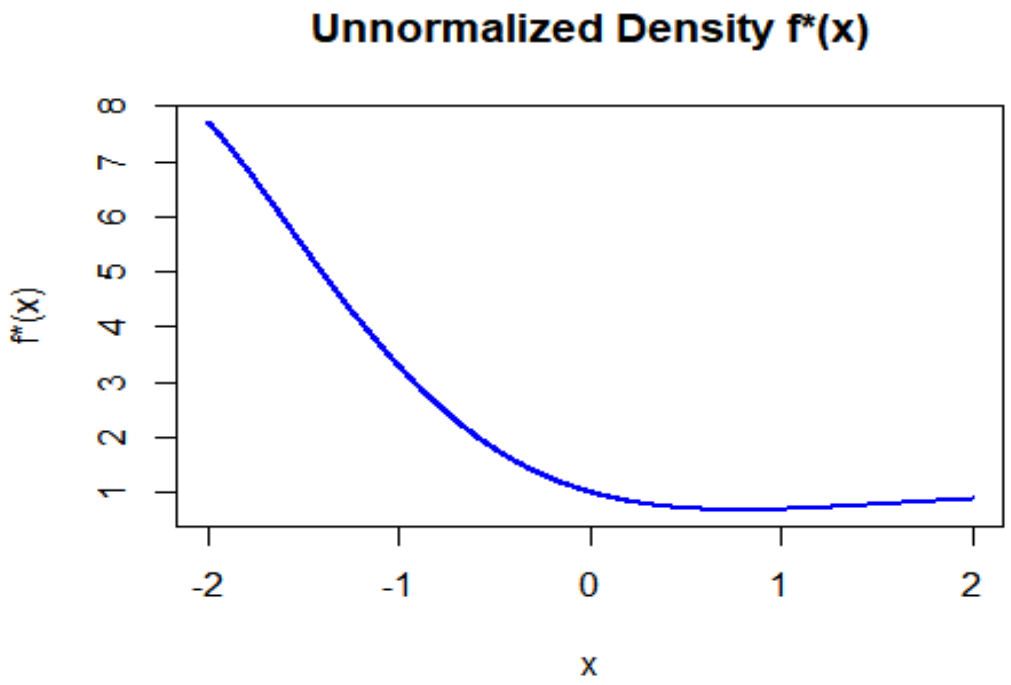
Question 2

(a) Plotting the (Not-Normalized) *f*

```r
# Define the unnormalized target function
f_unnormalized <- function(x) {
  -exp(-x) * sin(x) + 1
}

# Create a sequence of x values in (-2, 2)
x_vals <- seq(-2, 2, length.out = 1000)
y_vals <- f_unnormalized(x_vals)

# Plot the unnormalized density
plot(x_vals, y_vals, type = "l", col = "blue", lwd = 2,
     main = "Unnormalized Density f*(x)",
     xlab = "x", ylab = "f*(x)")
```



(b) Estimating the Normalizing Constant C via Monte Carlo

In this question, I estimate the normalizing constant C for the unnormalized density $f^*(x) = -e^{-x}\sin(x) + 1$ on the interval $[-2,2]$. I first note that

$$C = \int_{-2}^{2} f^*(x)\, dx.$$

To approximate this integral, I generate a large sample of x-values uniformly over $[-2,2]$. Since the probability density of a uniform variable on this interval is $1/4$ (because the length is 4), the expected value of $f^*(x)$ equals $\frac{1}{4}C$. Thus, I estimate C by computing the sample mean of $f^*(x)$ and then multiplying by 4:

$$\hat{C} = 4 \times \text{mean}\{f^*(x_i)\}.$$

Here, I used the fact that $I(x) = x$ to directly work with the sampled x-values.

```
set.seed(123)
N <- 100000  # Number of Monte Carlo samples
x_samples <- runif(N, min = -2, max = 2)  # Sample uniformly in (-2,2)
C_estimate <- 4 * mean(f_unnormalized(x_samples))  # 4 = length of interval
print(C_estimate)
```

```
## [1] 8.936518
```

(c) Metropolis–Hastings Algorithm with Exponential Kernel

-Use the Metropolis–Hastings algorithm with the proposal kernel

$$p(y) = \lambda e^{-\lambda y}, \quad y > 0,$$

For each $\lambda = 0.1, 1,$ and 10,obtain a Markov chain

We must implement the Metropolis–Hastings algorithm to generate samples from a target density defined on the interval $(-2,2)$ using a symmetric proposal. In our case, the unnormalized target density is

$$f^*(x) = -e^{-x}\sin(x) + 1,$$

and we use an exponential proposal with parameter λ. Because the exponential distribution is defined on $(0, \infty)$, I make the proposal symmetric by randomly assigning a positive or negative sign to the drawn step. (In other words, I generate a step δ and then use the mapping $I(\delta) = \delta$ along with a random sign so that the proposal becomes $y = x +$ sign $\times\ \delta$.)

I also ensure that each proposed value falls within the support $(-2,2)$; if it does not, I resample until it does. Finally, I compute the acceptance probability .since the proposal density is symmetric (i.e., the density for proposing y from x equals that for proposing x from y). Our goal is to simulate a Markov chain that converges to the target distribution, after which we use diagnostics—first a histogram (to check if the empirical density aligns with the true normalized density) and then a trace plot (to assess the chain's mixing and movement)—to evaluate the algorithm's performance.

```r
# Define the unnormalized target density (same as before)
f_target <- function(x) {
  -exp(-x) * sin(x) + 1
}

# Metropolis-Hastings function using a symmetric proposal
MH_exponential <- function(x, lambda) {
  repeat {
    # Sample a step from an exponential distribution and assign a random sign
    delta <- rexp(1, rate = lambda)
    sign <- sample(c(-1, 1), 1)
    y <- x + sign * delta
    # Accept only if y is within the support (-2, 2)
```

```
    if (y > -2 && y < 2) break
  }
  # Compute acceptance probability (proposal is symmetric so the ratio
simplifies)
  a <- min(1, f_target(y) / f_target(x))
  if (runif(1) < a) {
    return(y)  # Accept the move
  } else {
    return(x)  # Reject and stay at x
  }
}

# Function to simulate a Metropolis-Hastings chain
simulate_MH_chain <- function(lambda, n_iter = 5000, start = 0) {
  chain <- numeric(n_iter)
  chain[1] <- start
  for (i in 2:n_iter) {
    chain[i] <- MH_exponential(chain[i - 1], lambda)
  }
  return(chain)
}
```

Here, I used the normalized density $f_{\text{normalized}}(x) = \frac{f^*(x)}{C}$ (where C is the estimated normalizing constant) because it represents the true probability density function—its area under the curve equals 1. When plotting the histogram of the Markov chain samples, we want to compare our empirical distribution to a properly scaled theoretical density. Using $f_{\text{normalized}}$ ensures that the overlay is mathematically consistent with the probability distribution we aim to sample from and assess via the histogram.

```
# Use the previously estimated C_estimate from part (b)
f_normalized <- function(x) {
  f_target(x) / C_estimate
}
```

Now, simulate and plot for each lambda:

```
# Set up parameters for plotting
lambda_values <- c(0.1, 1, 10)
par(mfrow = c(1, 2))

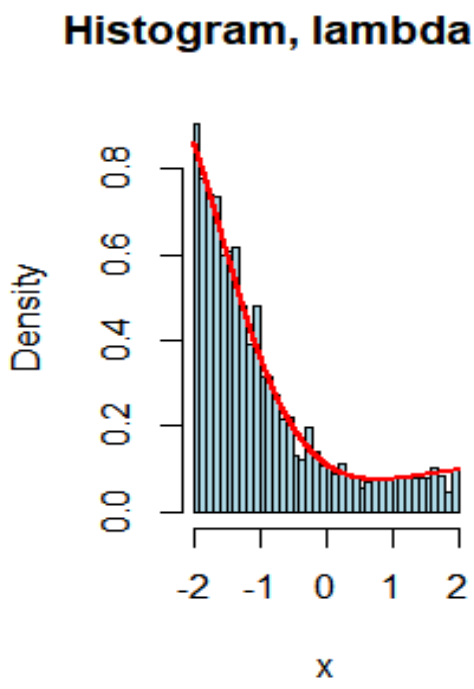
# Loop over each lambda value
for (lambda in lambda_values) {
  # Simulate the chain
  set.seed(123)
  chain <- simulate_MH_chain(lambda, n_iter = 5000, start = 0)

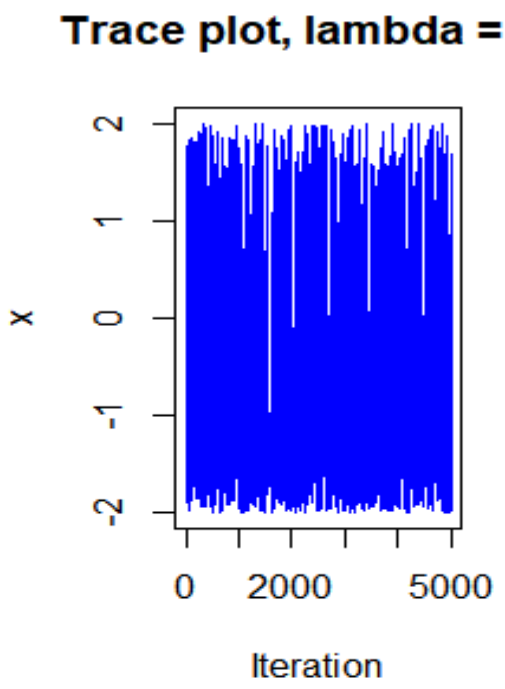
  # Plot the histogram of the chain
  hist(chain[1001:5000], breaks = 30, probability = TRUE,
       main = paste("Histogram, lambda =", lambda),
```

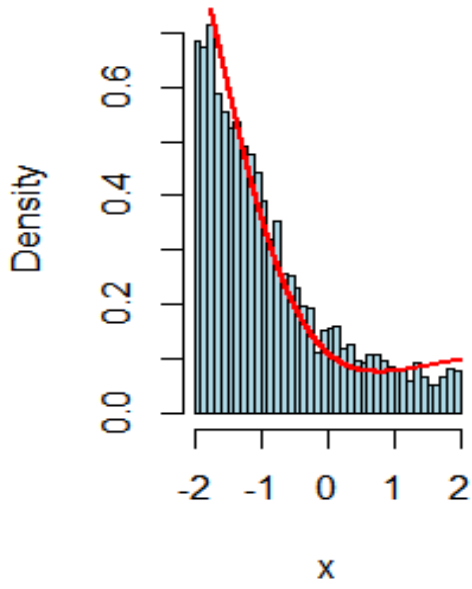
```r
       xlab = "x", col = "lightblue", border = "black")

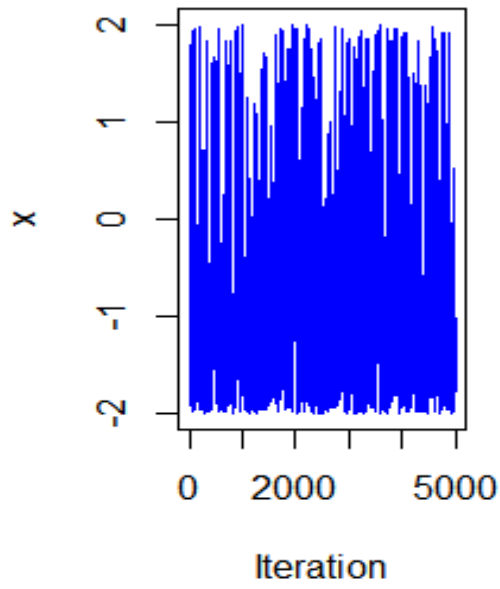
  # Add the true normalized density curve
  x_curve <- seq(-2, 2, length.out = 1000)
  lines(x_curve, f_normalized(x_curve), col = "red", lwd = 2)

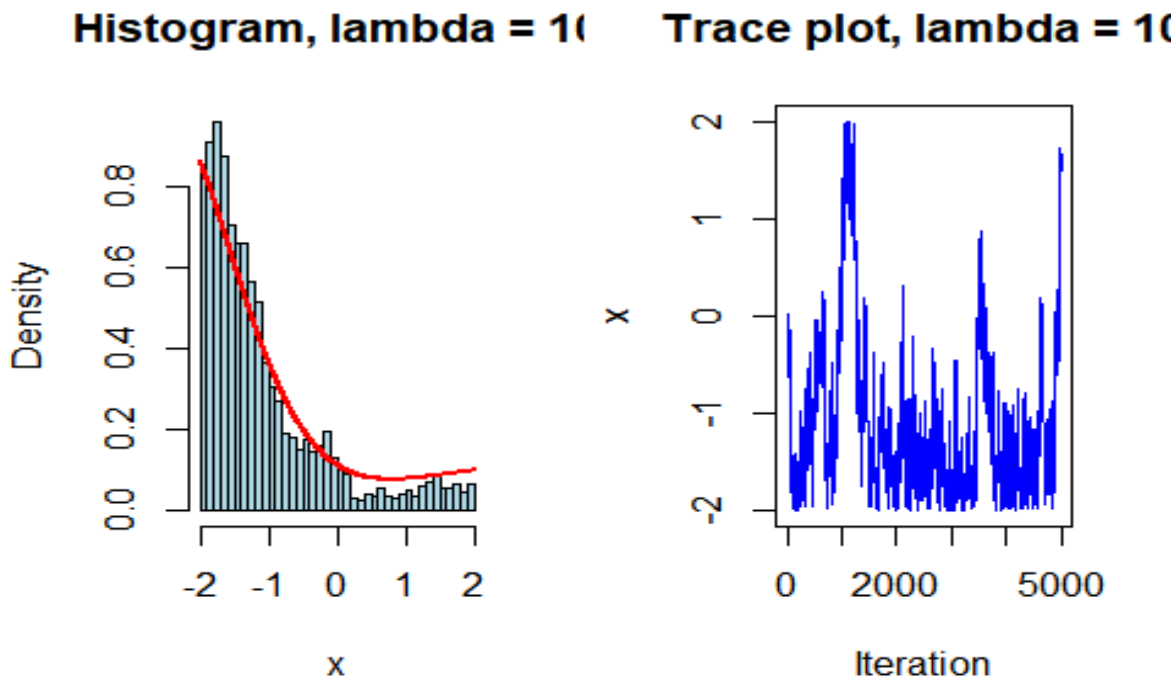
  # Plot the trace (path) of the chain
  plot(chain, type = "l", col = "blue",
       main = paste("Trace plot, lambda =", lambda),
       xlab = "Iteration", ylab = "x")
}
```


Histogram, lambda = 0.
Density
0.0
0.2
0.4
0.6
0.8
-2
-1
0
1
2
x


Trace plot, lambda = 0.
x
-2
-1
0
1
2
0
2000
5000
Iteration


Histogram, lambda = 1
Density
0.0
0.2
0.4
0.6
-2
-1
0
1
2
x


Trace plot, lambda = 1
x
-2
-1
0
1
2
0
2000
5000
Iteration



Commentary on the Histograms

$$\lambda = 0.1$$

The chain can jump broadly across $(-2,2)$. We see that the histogram still approximates the true density (the red curve), but some regions may be over- or under-sampled if the chain frequently jumps out of bounds and reverts to the current state. Overall, it captures the general shape but with moderate "roughness" because of these large jumps.

$$\lambda = 1$$

Typically, a moderate step size yields a better balance between acceptance rate and exploration. You can see the histogram fits the red density curve more smoothly, indicating a good coverage of the interval $(-2,2)$ without too many rejections or "overshoots."

$$\lambda = 10$$

Now the average step is 0.1, which is **small**. Because steps are small, the acceptance rate is higher, but the chain can move slowly around the space. Over 5000 iterations, it still approximates the target distribution, but the histogram may

show slightly heavier sampling in certain local regions if the chain has not traveled enough to reach the far ends frequently. However, from the plot, we see it still tracks the red curve reasonably well.

Commentary on the Trace Plots

I also include trace plot here because In **advanced Monte Carlo** (particularly Markov chain Monte Carlo), it is **not enough** to only look at the final histogram. A chain might produce a histogram that looks acceptable but could be "stuck" for many iterations, or fail to explore certain regions thoroughly. The **trace plot** (also called a path or time-series plot) shows how the Markov chain moves at each iteration, giving us insight into **mixing** and **autocorrelation**.

$\lambda = 0.1$:
- Large jumps (mean step size ~ 10) cause the chain to bounce widely within $[-2,2]$. We see in the trace that it frequently hits near -2 or +2 (attempting large moves). The chain covers the entire domain, but it can produce repeated rejections whenever a proposed step goes outside $(-2,2)$.

$\lambda = 1$:
- Moderate jumps yield a trace that systematically visits all regions without too many rejections or prolonged sticking. The chain moves fairly smoothly across $(-2,2)$, which usually results in better mixing.

$\lambda = 10$:
- Small jumps (mean step size ~ 0.1) lead to high acceptance, but the chain moves slowly across the space. The trace plot typically shows shorter excursions up and down, reflecting local exploration. It can take longer for the chain to traverse from one boundary to the other, so we see less "global" movement in any given number of iterations.

Hence, the trace plot **complements** the histogram by revealing how quickly or slowly the chain explores the domain and whether it might be "stuck" for long periods or jumping too far.

Question 3

Question 3

our goal here is to derive the posterior density $f_{\Lambda|X}(\lambda|x)$ using Bayes' rule.

By Bayes's theorem, the posterior density is given by:

$$f_{\Lambda|X}(\lambda|x) = \frac{f_{X|\Lambda}(x|\lambda) f_{\Lambda}(\lambda)}{\int_0^\infty f_{X|\Lambda}(x|\lambda') f_{\Lambda}(\lambda') d\lambda'}$$

Then we substitute the given expressions:

$f_{X|\Lambda}(x|\lambda) f_{\Lambda}(\lambda) = [\lambda e^{-\lambda x}] \times [\beta e^{-\beta\lambda}] = \beta\lambda e^{-\lambda x} e^{-\beta\lambda}$

Then we combine the exponential terms: $\beta\lambda e^{-\lambda(x+\beta)}$

Thus the unnormalized posterior is: $\propto \beta\lambda e^{-\lambda(x+\beta)}$

The determinator is the integral over all possible λ:

$\int_0^\infty f_{X|\Lambda}(x|\lambda) f_{\Lambda}(\lambda) d\lambda = \int_0^\infty \beta\lambda e^{-\lambda(x+\beta)} d\lambda$

let $a = x+\beta$. Then, the integral becomes:

$\beta \int_0^\infty \lambda e^{-a\lambda} d\lambda$.

Recall the formula for the Gamma integral:

$\int_0^\infty \lambda^n e^{-a\lambda} d\lambda = \frac{\Gamma(n+1)}{a^{n+1}}$, with $n=1$.

Knowing that $\Gamma(2) = 1! = 1$, we have:

$\int_0^\infty \lambda e^{-a\lambda} d\lambda = \frac{1}{a^2} = \frac{1}{(x+\beta)^2}$

Now we devide the unnormaliged posterior by normalization constant:

$f_{\Lambda|X}(\lambda|x) = \frac{\beta\lambda e^{-\lambda(x+\beta)}}{\beta/(x+\beta)^2} = (x+\beta)^2 \lambda e^{-\lambda(x+\beta)}$

This expression is the posterior density of Λ given x

Question 3

The gamma density with shape parameter k and rate parameter θ is given by:

$g(\lambda; k; \theta) = \frac{\theta^k}{\Gamma(k)} \lambda^{k-1} e^{-\theta\lambda}$, $\lambda > 0$

Comparing our expression $f_{\Lambda|X}(\lambda|x) = (x+\beta)^2 \lambda e^{-\lambda(x+\beta)}$

So we identify.

- The exponent on λ is 1, which means λ^{2-1}; hence the shape parameter $k=2$
- The rate parameter is $\theta = x+\beta$

Since $\Gamma(2) = 1$ the normaliged Gamma density is:

$g(\lambda; 2, x+\beta) = (x+\beta)^2 \lambda e^{-\lambda(x+\beta)}$

Thus the posterior density is exactly a gamma density with shape 2 and rate $x+\beta$: $f_{\Lambda|X}(\lambda|x) = (x+\beta)^2 \lambda e^{-\lambda(x+\beta)}$, $\lambda > 0$